

Robot Learning Engineer Master Curriculum

Long-Term Roadmap 2026-2028 | From Mechanical Engineering Student to Robot Learning / Embodied AI Engineer

This roadmap is optimized for high-value roles in robotics + AI: Robot Learning Engineer, Embodied AI Engineer, Robotics ML Engineer, Autonomy ML Engineer, Robotics Research Engineer, and Physical AI Engineer. It does not promise a specific package, company, or university admit. It is designed to maximize skill depth, project evidence, research readiness, and interview readiness.

Operating Principle

- Primary target: Robot Learning / Embodied AI Engineer with strong software engineering and robotics fundamentals.
- Core stack: Python, C++, DSA, PyTorch, ROS 2, Linux, Git, computer vision, control, planning, simulation, reinforcement learning, imitation learning, and research reading.
- Proof system: every week must produce GitHub commits, written notes, solved problems, and at least one visible deliverable.
- CGPA rule: protect your degree. Strong CGPA plus strong projects is far better than projects alone.
- Safety rule: certificates are secondary. Projects, code, research reports, internships, publications, and open-source evidence are primary.

Two-Year Skill Tree

Layer	What you must become good at	Evidence by graduation
Programming	Python, Modern C++, Git, Linux, Bash, Docker, clean code	Repos with tested Python/C++ robotics software
CS/DSA	Arrays, strings, hash maps, trees, graphs, DP, complexity	300-500 quality problems with notes
Math	Linear algebra, probability, calculus, optimization, geometry, quaternions	Math notebooks and robotics derivations
AI/ML	Supervised ML, deep learning, PyTorch, CNNs, transformers	Vision model and PyTorch training projects
Robotics	ROS 2, kinematics, control, SLAM, planning, navigation, manipulation	Mobile robot + arm + UAV projects
Robot Learning	RL, imitation learning, offline RL, diffusion policy, VLA ideas	Simulation project with learned policy
Career	Resume, GitHub, LinkedIn, interview prep, outreach, SOP	Internships, applications, strong portfolio

The 12 Volumes

Each volume has one primary learning direction, one main outcome, and one proof-of-work deliverable. We will use weekly packets to execute these volumes.

Volume 0 - Orientation and Success System

- Study: Build the execution system: weekly planning, daily reports, GitHub structure, notes, review cycles, CGPA protection, and career direction.
- Deliverable: One Notion/Markdown dashboard, GitHub repo, weekly report template, habit tracker.

Volume 1 - Programming Foundations

- Study: Python, Modern C++, Git, Linux, Bash, VS Code, debugging, testing, packaging basics.
- Deliverable: Python mini-projects, C++ mini-projects, clean GitHub commit history.

Volume 2 - Computer Science and DSA

- Study: Data structures, algorithms, OOP, OS basics, DBMS basics, networks basics, coding interviews.
- Deliverable: DSA notebook, LeetCode/Codeforces progress, C++ implementations.

Volume 3 - Mathematics for Robotics and AI

- Study: Linear algebra, calculus, probability, statistics, optimization, geometry, transforms, quaternions.
- Deliverable: Math-for-robotics notebooks and solved derivations.

Volume 4 - Machine Learning and Deep Learning

- Study: ML fundamentals, PyTorch, CNNs, transformers, model training, evaluation, experiment tracking.
- Deliverable: End-to-end PyTorch project and documented experiments.

Volume 5 - Robotics Fundamentals

- Study: ROS 2, robot description, TF, sensors, kinematics, dynamics, control, motion planning, navigation.
- Deliverable: ROS 2 mobile robot stack with simulation and documentation.

Volume 6 - Computer Vision for Robotics

- Study: OpenCV, camera models, calibration, depth, pose estimation, detection, segmentation, tracking, 3D vision.
- Deliverable: Vision-guided robot perception project.

Volume 7 - Robot Learning and Embodied AI

- Study: Reinforcement learning, imitation learning, offline RL, behavior cloning, diffusion policies, VLA foundations.
- Deliverable: Learned policy in simulation with evaluation metrics.

Volume 8 - Simulation and Sim-to-Real

- Study: Gazebo, MuJoCo, Isaac Sim/Isaac Lab concepts, domain randomization, benchmarking, digital twins.
- Deliverable: Repeatable simulation benchmark with logs and plots.

Volume 9 - Hardware, Embedded Systems, and Sensors

- Study: Arduino/ESP32, Raspberry Pi/Jetson, IMU, cameras, motors, buses, sensor fusion, basic PCB awareness.
- Deliverable: Sensor + actuation prototype integrated with ROS 2.

Volume 10 - Research, Open Source, and Publications

- Study: Paper reading, reproducing results, writing reports, GitHub issues/PRs, conference awareness.
- Deliverable: One research-style report, one reproduction, one meaningful OSS contribution.

Volume 11 - Career, Internships, Placements, and MS

- Study: Resume, LinkedIn, GitHub, interview prep, internship strategy, MS SOP/LOR plan, outreach.
- Deliverable: Application-ready portfolio, tailored CVs, interview readiness.

Primary Resource Philosophy

For each topic, the rule is one primary course, one primary book, official documentation, and one proof project. Avoid collecting ten resources and finishing none.

Area	Primary course	Primary book	Practice / documentation
Python	Harvard CS50P	Python Crash Course, 3rd ed.	Python official tutorial
C++	learncpp.com structured path	A Tour of C++ / C++ Primer	cppreference
DSA	Princeton Algorithms I/II	Grokking Algorithms first, then CLRS/Algorithm Design	LeetCode + Codeforces
Math	MIT OCW Linear Algebra + Khan/3Blue1Brown for intuition	Mathematics for Machine Learning	NumPy notebooks
Deep Learning	DeepLearning.AI or fast.ai/PyTorch official tutorials	Deep Learning by Goodfellow et al.	PyTorch docs
Robotics	Modern Robotics	Modern Robotics: Mechanics, Planning, and Control	ROS 2 docs
Computer Vision	CS231n / OpenCV tutorials	Computer Vision: Algorithms and Applications	OpenCV docs
RL	Berkeley CS285	Reinforcement Learning: Sutton and Barto	CleanRL / Stable-Baselines3
Simulation	Official docs + projects	Robotics, Vision and Control	Gazebo, MuJoCo, Isaac docs

Semester-Level Milestones

Period	Main focus	Minimum outcomes
Jul-Dec 2026	Programming, DSA, Python/C++, Git/Linux, ML basics, ROS 2 basics	Strong repo system, 80-120 DSA problems, 2 mini-projects, 1 robotics demo
Jan-May 2027	PyTorch, computer vision, ROS 2 navigation, math	Vision model, ROS 2 robot project, 200+ DSA total
Jun-Aug 2027	Internship/research + advanced project	One serious research/project report, portfolio upgrade, applications
Sep-Dec 2027	Robot learning, RL/imitation learning, manipulation/simulation	Simulation policy project, paper reproduction, OSS contribution
Jan-May 2028	Placement/MS readiness, capstone, interviews	Final portfolio, tailored CV, interview readiness, SOP drafts if applying

Weekly Operating System

- Before the week begins: generate a full weekly PDF packet.
- Each day: follow the weekly packet, then send a Daily Completed Report.
- Every Sunday: weekly review, GitHub cleanup, next-week packet, and backlog adjustment.
- Every month: compile a monthly PDF volume with completed work, notes, and next targets.

Daily Completed Report

1. Hours studied:
2. Tasks completed:
3. GitHub commits:
4. Problems solved:

5. What was easy:
6. What was hard:
7. What I could not complete:
8. Questions for tomorrow:

Career Outcomes This Roadmap Supports

- On-campus software/AI roles: supported by DSA, C++, Python, CS fundamentals, and interview preparation.
- Off-campus robotics/AI roles: supported by ROS 2, PyTorch, computer vision, simulation, robot learning projects, and GitHub proof.
- Research internships: supported by reports, paper reading, reproducible experiments, and faculty outreach.
- MS applications: supported by CGPA, research experience, strong SOP story, recommendation letters, projects, and publications if they materialize.

Important honesty note: no curriculum can guarantee a core-level job, a specific company, or a specific MS admit. The objective is to build the kind of evidence and ability that strong candidates usually show.

Reference Links

- CS50P: <https://cs50.harvard.edu/python/>
- Python Tutorial: <https://docs.python.org/3/tutorial/>
- Pro Git: <https://git-scm.com/book/en/v2>
- Princeton Algorithms I: <https://www.coursera.org/learn/algorithms-part1>
- PyTorch Learn the Basics: <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>
- OpenCV Python Tutorials: https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- ROS 2 Humble Tutorials: <https://docs.ros.org/en/humble/Tutorials.html>
- Modern Robotics: <https://modernrobotics.northwestern.edu/>
- Berkeley CS285: <https://rail.eecs.berkeley.edu/deeprlcourse/>
- LeetCode: <https://leetcode.com/>